

Lab: Diffie-Hellman Using OpenSSL

Estimated Time: 30 minutes

Overview

In this lab, you will learn how to perform key exchange using the Diffie-Hellman (DH) algorithm with OpenSSL. You will also generate DH parameters, create private and public keys, and derive a shared secret key between two parties.

Learning objectives

By the end of this lab, you will be able to:

- Generate Diffie-Hellman parameters
- Create and exchange public keys
- Derive a shared secret key

Prerequisites (Optional)

You should:

- Be familiar with the Linux command line.
 - Have a basic understanding of cryptographic key exchange concepts.
-

Step 1: Generate Diffie-Hellman Parameters

Run the below command to generate the DH parameters required for the key exchange.

```
openssl dhparam -out dhparam.pem 2048
```

Note: The key generation process takes some time. Please wait for the process to complete.

Command breakdown:

- `openssl dhparam`: Generates DH parameters.
- `-out dhparam.pem`: Specifies the output file for the parameters.
- `2048`: Indicates the size of the DH parameters in bits.

Run the below command to verify the generated file.

```
cat dhparam.pem
```

Step 2: Generate a Private Key for Party 1

Run the below command to use the DH parameters to generate a private key for Party 1.

```
openssl genpkey -paramfile dhparam.pem -out private1.pem
```

Command breakdown:

- `openssl genpkey`: Generates a private key.
 - `-paramfile dhparam.pem`: Specifies the DH parameter file.
 - `-out private1.pem`: Saves the private key to a file.
-

Step 3: Extract the Public Key for Party 1

Run the below command to extract the public key for Party 1:

```
openssl pkey -in private1.pem -pubout -out public1.pem
```

Command breakdown:

- `openssl pkey`: Manages private and public keys.
 - `-in private1.pem`: Specifies the private key file.
 - `-pubout`: Extracts the public key.
 - `-out public1.pem`: Saves the public key to a file.
-

Step 4: Generate a Private Key for Party 2

Repeat above Steps 2 and 3 for Party 2 to generate their private and public keys:

```
openssl genpkey -paramfile dhparam.pem -out private2.pem
openssl pkey -in private2.pem -pubout -out public2.pem
```

Step 5: Derive the Shared Secret Key

Follow the below steps to derive the shared secret key.

Step A: Party 1 derives the shared secret

To derive the shared secret, Party 1 uses their private key and Party 2's public key using the below command.

```
openssl pkeyutl -derive -inkey private1.pem -peerkey public2.pem -out shared_secret1.bin
```

Step B: Party 2 derives the shared secret

To derive the shared secret, Party 2 uses their private key and Party 1's public key using the below command.

```
openssl pkeyutl -derive -inkey private2.pem -peerkey public1.pem -out shared_secret2.bin
```

Command breakdown:

- `openssl pkeyutl -derive`: Derives a shared secret using private and peer public keys.
- `-inkey`: Specifies the private key file.

- -peerkey: Specifies the peer's public key file.
- -out: Specifies the output file for the shared secret.

Step 6: Verify the Shared Secret

Use the below command to compare the derived shared secret files from both parties.

```
diff shared_secret1.bin shared_secret2.bin
```

If the files are identical, the Diffie-Hellman key exchange was successful.

Exercises

Exercise 1: Generate and Verify Shared Secrets

Objective:

Understand how the Diffie-Hellman (DH) key exchange generates a shared secret between two parties.

Task:

1. Generate a DH parameter file (for example, dhparam.pem).
2. Simulate two parties (Alice and Bob) by generating private keys and public keys for each party.
3. Derive the shared secret on both sides and confirm they match.

Hint:

- Generate DH parameters using:

```
openssl dhparam -out dhparam.pem 2048
```

- Create private and public keys for Alice and Bob using:

```
openssl genpkey -paramfile dhparam.pem -out alice_private.pem
openssl pkey -in alice_private.pem -pubout -out alice_public.pem
openssl genpkey -paramfile dhparam.pem -out bob_private.pem
openssl pkey -in bob_private.pem -pubout -out bob_public.pem
```

- Derive the shared secret for Alice:

```
openssl pkeyutl -derive -inkey alice_private.pem -peerkey bob_public.pem -out shared_secret_alice.bin
```

- Derive and compare Bob's shared secret with Alice's to ensure they match.
-

Exercise 2: Experiment with Different DH Parameter Sizes

Objective:

Explore how different parameter sizes affect the security and performance of the DH key exchange.

Task:

1. Generate DH parameters with sizes 1024, 2048, and 4096 bits.
2. Perform a DH key exchange for each parameter size and measure the time taken.
3. Compare generated shared secrets.

Hint:

- Use the -out and -5 options with different sizes:

```
openssl dhparam -out dh1024.pem 1024
openssl dhparam -out dh2048.pem 2048
openssl dhparam -out dh4096.pem 4096
```

- Measure the time to generate keys using time:

```
time openssl genpkey -paramfile dh2048.pem -out private_key.pem
```

- Verify that shared secrets match for each parameter size using sha256sum.
-

Exercise 3: Simulate a Man-in-the-Middle (MITM) Attack

Objective:

Understand the vulnerabilities of the DH key exchange when used without authentication.

Hint:

Step 1: Generate DH parameters

1. Create a DH parameter file (dhparam.pem):

```
openssl dhparam -out dhparam.pem 2048
```

Step 2: Generate keys for Alice, Bob, and Eve

1. Alice's private and public keys:

```
openssl genpkey -paramfile dhparam.pem -out alice_private.pem  
openssl pkey -in alice_private.pem -pubout -out alice_public.pem
```

2. Bob's private and public keys:

```
openssl genpkey -paramfile dhparam.pem -out bob_private.pem  
openssl pkey -in bob_private.pem -pubout -out bob_public.pem
```

3. Eve's private and public keys:

```
openssl genpkey -paramfile dhparam.pem -out eve_private.pem
```

```
openssl pkey -in eve_private.pem -pubout -out eve_public.pem
```

Step 3: Intercept and derive shared secrets

Case 1: Alice → Eve (Intercepted):

Eve intercepts Alice's public key (alice_public.pem) and used it to derive a shared secret with her own private key (eve_private.pem).

1. Eve derived a shared secret with Alice:

```
openssl pkeyutl -derive -inkey eve_private.pem -peerkey alice_public.pem -out shared_secret_eve_alice.bin
```

Case 2: Bob → Eve (Intercepted):

Eve intercepts Bob's public key (bob_public.pem) and used it to derive a shared secret with her own private key (eve_private.pem).

2. Eve derives a shared secret with Bob:

```
openssl pkeyutl -derive -inkey eve_private.pem -peerkey bob_public.pem -out shared_secret_eve_bob.bin
```

Step 4: Verify shared secrets

1. Hash the derived shared secrets to ensure confidentiality:

```
openssl dgst -sha256 shared_secret_eve_alice.bin  
openssl dgst -sha256 shared_secret_eve_bob.bin
```

2. Notice that the shared secret between Eve and Alice is different from the one between Eve and Bob. However, Alice and Bob do not share a common secret, breaking the intended secure communication.

Step 5: Observe the MITM attack

1. Eve can decrypt messages from Alice using her shared secret (`shared_secret_eve_alice.bin`) and re-encrypt them for Bob using her shared secret with Bob (`shared_secret_eve_bob.bin`).
2. This demonstrates how Eve can relay and manipulate communication without either Alice or Bob realizing.

Conclusion

In this lab, you successfully performed a Diffie-Hellman key exchange using OpenSSL. You've generated DH parameters, created private and public keys, and derived a shared secret. Additionally, you learned how two parties can securely exchange keys over an insecure channel. These principles are foundational in many secure communication protocols.

Author(s)

Dr. Manish Kumar

© IBM Corporation. All rights reserved.